

CHAPTER 6

FRONT-END AND BACK-END CODE OF CLASSROOM & LAB UTILIZATION OPTIMIZATION SYSTEM

6.1 Front-End Module codes

The front-end of the application is built using React and TypeScript. Below is the main application routing file (`App.tsx`) which handles protected routes, context integration, and navigation logic.

src/App.tsx:

```
import { Routes, Route, Navigate } from 'react-router-dom';
import { useAuth } from './contexts/AuthContext';
import LandingPage from './components/LandingPage';
import AuthPage from './components/auth/AuthPage';
import DashboardLayout from './components/DashboardLayout';

function ProtectedRoute({ children }: { children: React.ReactNode }) {
  const { isAuthenticated, loading } = useAuth();
  if (loading) {
    return (
      <div className="min-h-screen flex items-center justify-center bg-gray-50">
        <div className="flex items-center gap-3">
          <div className="w-8 h-8 border-3 border-blue-200 border-t-blue-600 rounded-full
animate-spin" />
          <span className="text-gray-500 font-medium">Loading...</span>
        </div>
      </div>
    );
  }
  return isAuthenticated ? <>{children}</> : <Navigate to="/auth" replace />;
}

function App() {
  return (
    <Routes>
      <Route path="/" element={<LandingPage />} />
      <Route path="/auth" element={<AuthPage />} />
      <Route
        path="/app/*"
        element={
```

```

    <ProtectedRoute>
      <DashboardLayout />
    </ProtectedRoute>
  }
/>
  <Route path="*" element={<Navigate to="/" replace /> } />
</Routes>
);
}

export default App;

```

6.2 Database connectivity

The back-end server connects to the MySQL database using the `mysql2` promise-based API. A connection pool is established to manage concurrent database queries, transactions, and row-level locking efficiently.

server/db.js:

```

import { createPool } from 'mysql2/promise';
import dotenv from 'dotenv';
import path from 'path';
import { fileURLToPath } from 'url';

const __dirname = path.dirname(fileURLToPath(import.meta.url));
dotenv.config({ path: path.join(__dirname, '.env') });

// Ensure your .env file in the server/ directory contains these variables:
// DB_HOST, DB_USER, DB_PASSWORD, DB_NAME (default: classroom_utilization_db)
const pool = createPool({
  host: process.env.DB_HOST || 'localhost',
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || '',
  database: process.env.DB_NAME || 'classroom_utilization_db',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

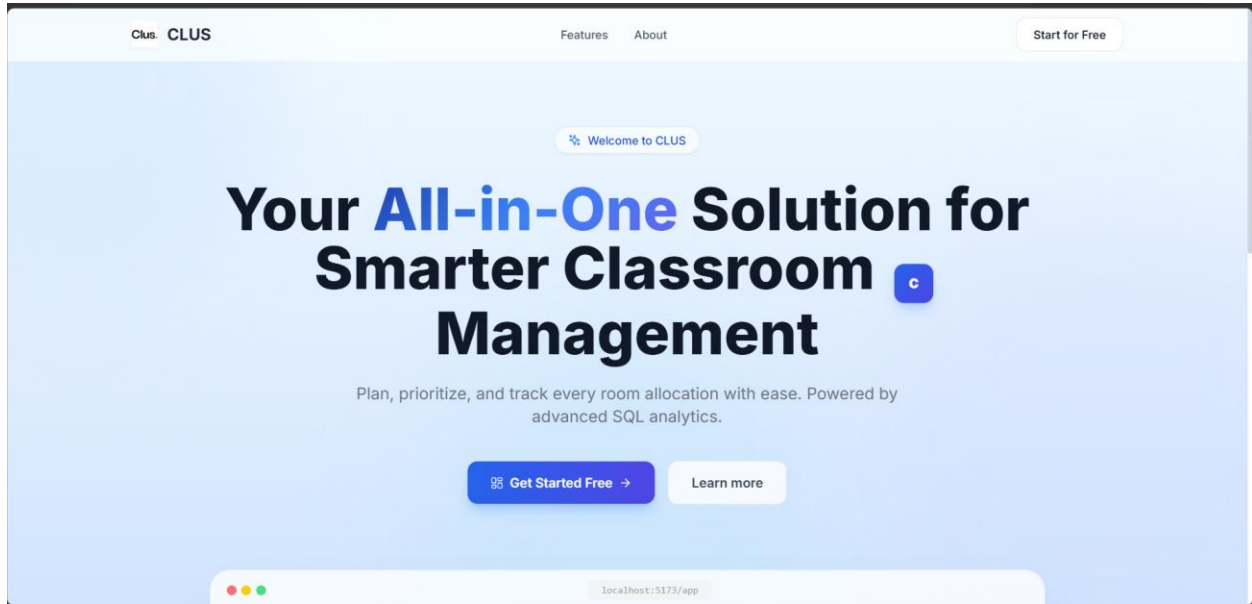
export default pool;

```

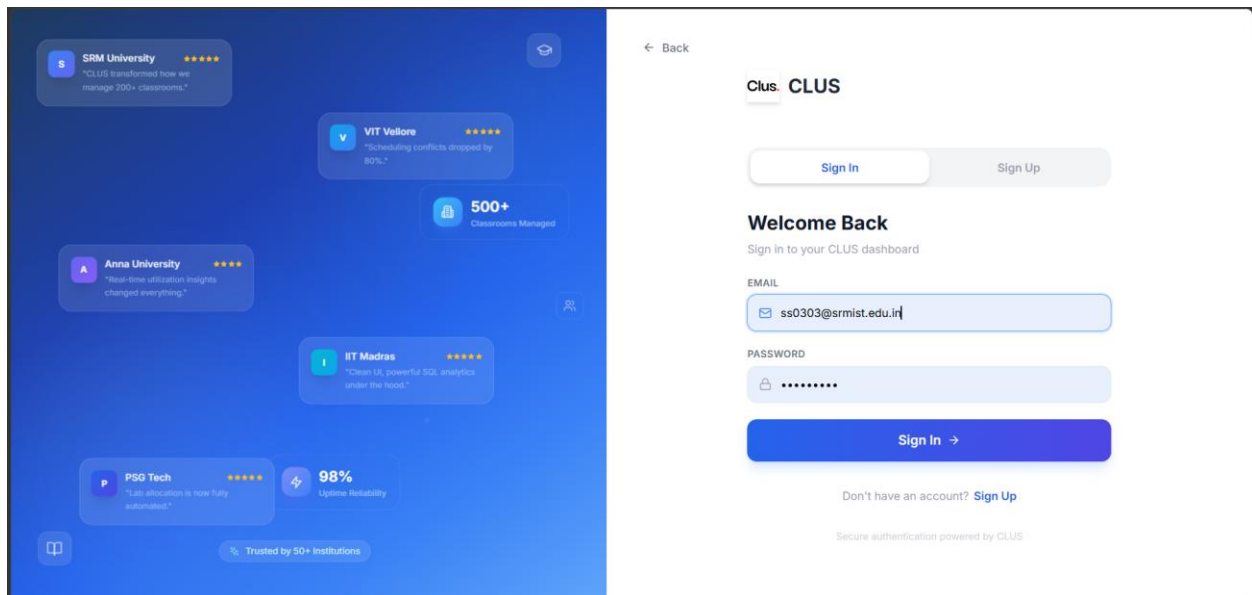
CHAPTER 7

RESULTS AND DISCUSSIONS

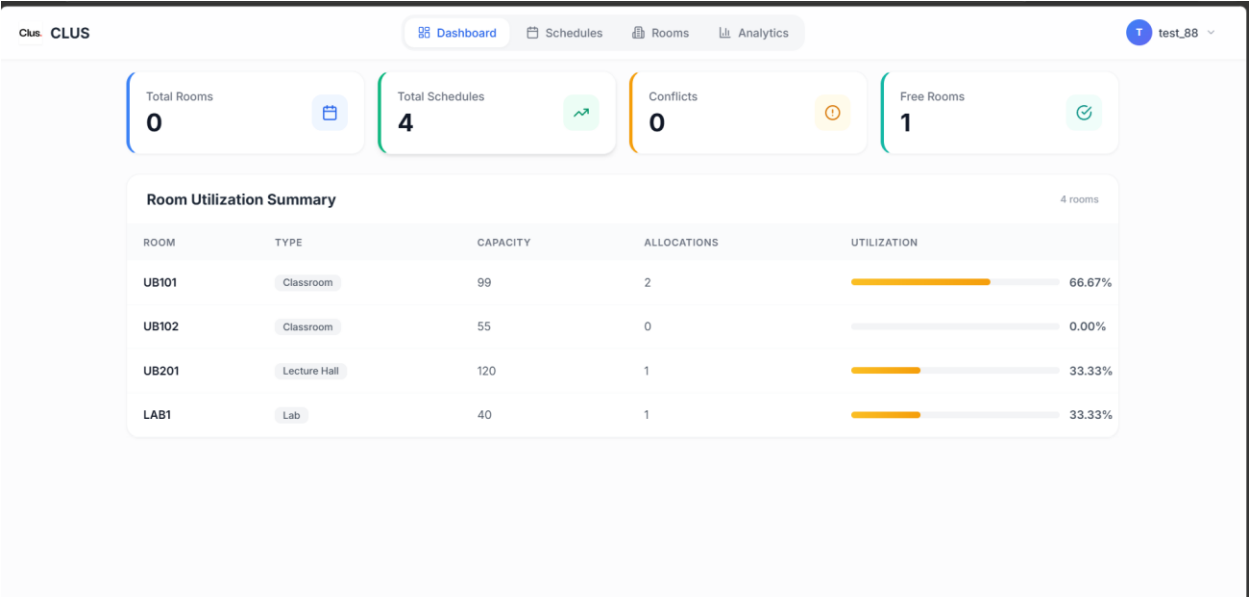
7.1 Screenshots of front-end



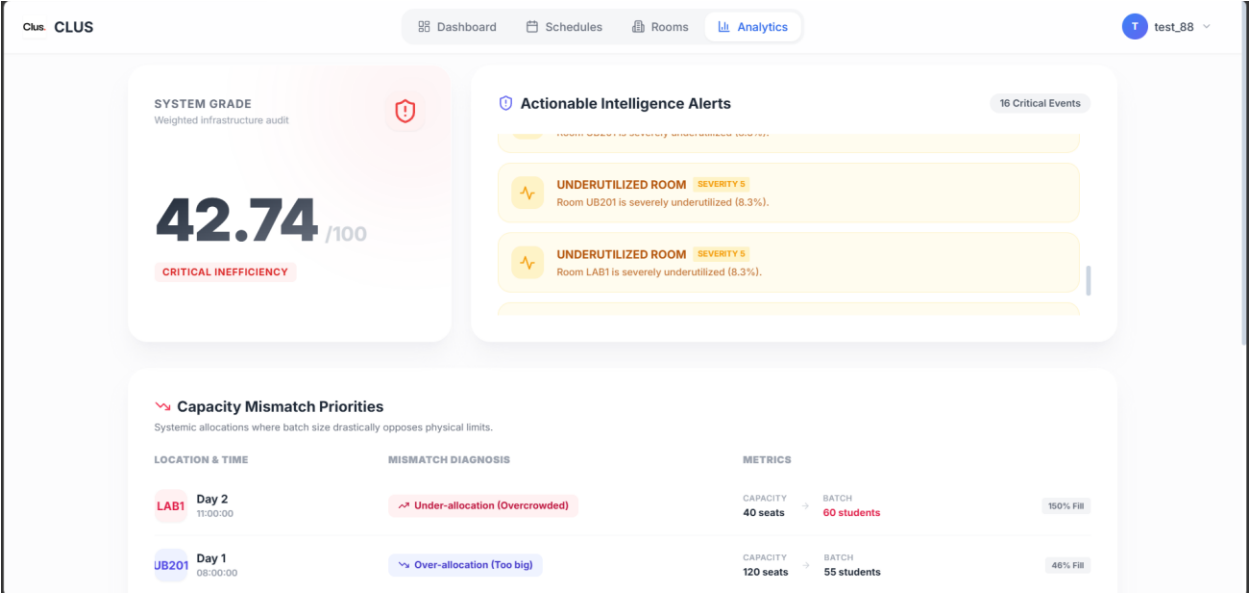
Chapter 7. Figure 7.1: Landing Page of the Application



Chapter 7. Figure 7.2: User Authentication Interface



Chapter 7. Figure 7.3: Main Dashboard showing Room Allocations and Utilization metrics



Chapter 7. Figure 7.4: Analytics tab showing course load and room saturation data

7.2 Screenshots of Database

```
mysql> use classroom_utilization_db
Database changed
mysql> show tables;
+-----+
| Tables_in_classroom_utilization_db |
+-----+
| actionableanalyticssignals
| batch
| capacitymismatchanalysis
| course
| course_schedule
| department
| resource
| room
| room_resource
| room_type_lookup
| temporalstressindex
| time_slot
| unifiedutilizationview
| users
| utilizationimbalance
| view_detailed_schedule
| view_room_utilization
| wastedcapacityview
+-----+
18 rows in set (0.02 sec)
```

Chapter 7. Figure 7.5: Database Tables in MySQL

```
mysql> SELECT
->     v.room_number,
->     r.room_type,
->     v.capacity,
->     v.slots_used,
->     get_utilization_percent(r.room_id) AS utilization_percentage
-> FROM View_Room_Utilization v
-> JOIN Room r ON v.room_number = r.room_number;
+-----+-----+-----+-----+-----+
| room_number | room_type | capacity | slots_used | utilization_percentage |
+-----+-----+-----+-----+-----+
| UB101      | Classroom | 99      | 2          | 66.67                 |
| UB102      | Classroom | 55      | 0          | 0.00                  |
| UB201      | Lecture Hall | 120     | 1          | 33.33                 |
| LAB1       | Lab       | 40      | 1          | 33.33                 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Chapter 7. Figure 7.6: Results of Utilization Percentage View

```
mysql> -- Attempting to book a different class into an already occupied room & time slot
mysql> INSERT INTO Course_Schedule (course_id, batch_id, room_id, slot_id)
      -> VALUES (103, 203, 301, 101);
ERROR 1644 (45000): Double Booking Error: The room is already occupied for this time slot!
mysql>
```

Chapter 7. Figure 7.7: Trigger Execution preventing double-booking

References

- [1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2015.
- [2] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. McGraw-Hill Education, 2019.
- [3] "MySQL 8.0 Reference Manual," Oracle Corporation. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>. [Accessed: May 2026].
- [4] "React Documentation," Meta Platforms, Inc. [Online]. Available: <https://reactjs.org/docs/getting-started.html>. [Accessed: May 2026].
- [5] "Express Node.js web application framework," OpenJS Foundation. [Online]. Available: <https://expressjs.com/>. [Accessed: May 2026].